



GUIA DE ESTUDOS

Guia - Tecnologias, Carreiras e Fundamentos da Engenharia de Software

Material introdutório sobre áreas, tecnologias e bases da Engenharia de Software

Marcelo R. Chamorro

Preparação para Engenharia de Software





Sobre este PDF

Visão geral do conteúdo

Este material apresenta uma visão introdutória e organizada sobre temas centrais da área de Engenharia de Software. Ao longo do PDF, o leitor encontrará explicações sobre papéis profissionais em tecnologia, fundamentos de linguagens de programação, paradigmas de desenvolvimento, arquitetura de software, APIs, bancos de dados, versionamento, testes, boas práticas e outros conceitos que ajudam a compreender como sistemas são planejados, construídos e evoluídos.

A proposta é servir como um guia de referência para quem deseja construir uma base sólida antes de aprofundar estudos mais técnicos. O conteúdo combina noções conceituais e visão de mercado, oferecendo um panorama útil tanto para iniciantes quanto para quem está retomando os estudos na área de tecnologia.

Em resumo, este PDF foi pensado para apoiar a compreensão do ecossistema de software: quais são as principais tecnologias, como as funções da área se relacionam, quais fundamentos estruturam o desenvolvimento de sistemas e por que esses conhecimentos são importantes na formação de um futuro profissional de Engenharia de Software.

1. ENGENHARIA DE SOFTWARE

A Engenharia de Software é uma disciplina da computação que se preocupa com o desenvolvimento sistemático, controlado e organizado de software. Diferente da programação isolada, ela envolve um conjunto de práticas, processos e princípios que permitem construir sistemas complexos de forma sustentável ao longo do tempo.

Ao desenvolver um software simples, uma única pessoa pode escrever código que resolve um problema específico. No entanto, quando o sistema cresce — envolvendo múltiplos usuários, regras de negócio complexas, integração com outros sistemas e necessidade de evolução constante — torna-se essencial adotar uma abordagem estruturada. É nesse contexto que a Engenharia de Software se torna fundamental.

Ela busca responder questões como:

- Como garantir que o software continue funcionando após mudanças?
- Como organizar o código para facilitar manutenção?
- Como permitir que múltiplos desenvolvedores trabalhem juntos?
- Como garantir desempenho e escalabilidade?

O ciclo de vida do software normalmente inclui:

- levantamento de requisitos
- análise do problema
- projeto da solução (design)
- implementação
- testes
- implantação (deploy)
- manutenção

Um dos principais objetivos da Engenharia de Software é reduzir a complexidade inerente ao desenvolvimento de sistemas, criando métodos que tornem o processo mais previsível, seguro e eficiente.

2. FUNÇÕES NA ÁREA DE TECNOLOGIA

O desenvolvimento de software moderno é um esforço coletivo. Diferentes profissionais atuam em conjunto para transformar uma ideia em um sistema funcional.

Desenvolvedor de Software

O desenvolvedor é responsável por escrever código e implementar funcionalidades. Ele traduz requisitos e regras de negócio em instruções que o computador pode executar.

Dependendo da área de atuação, pode se especializar em:

- Front-end: interface do usuário
- Back-end: lógica de negócio e manipulação de dados
- Full-stack: ambos

Seu trabalho envolve:

- escrever código
- corrigir erros
- integrar sistemas
- trabalhar com APIs e bancos de dados

Engenheiro de Software

O engenheiro de software possui uma visão mais ampla do sistema. Ele não apenas implementa, mas também projeta e estrutura soluções.

Suas responsabilidades incluem:

- definição de arquitetura
- escolha de tecnologias
- análise de desempenho
- garantia de qualidade do sistema

Enquanto o desenvolvedor foca mais na execução, o engenheiro foca na sustentabilidade do software.

Tech Lead

O Tech Lead é o líder técnico da equipe. Ele orienta decisões técnicas e garante que o time siga boas práticas.

Atua em:

- revisão de código
- definição de padrões
- suporte técnico à equipe
- tomada de decisões arquiteturais

Além disso, serve como ponte entre a equipe técnica e outras áreas da empresa.

DevOps Engineer (explicação aprofundada)

O DevOps Engineer atua na integração entre desenvolvimento e operações. Esse papel surgiu para resolver um problema comum: a separação entre quem desenvolve software e quem o mantém em funcionamento.

Historicamente, desenvolvedores criavam sistemas e os “entregavam” para outra equipe cuidar da implantação. Isso gerava conflitos:

- desenvolvedores queriam velocidade
- operações queriam estabilidade

DevOps surge como uma abordagem que une esses dois mundos.

Principais responsabilidades

Automação

- criação de pipelines de CI/CD
- automação de testes
- automação de deploy

Infraestrutura como código

- servidores definidos por código
- ambientes reproduzíveis

Containerização

- uso de Docker para empacotar aplicações

Orquestração

- uso de Kubernetes para gerenciar containers

Monitoramento

- análise de logs
- métricas de desempenho
- alertas

Objetivo do DevOps

- reduzir falhas
- aumentar velocidade de entrega
- garantir estabilidade

DevOps não é apenas uma função, mas uma cultura baseada em colaboração e automação.

QA (Quality Assurance)

O QA é responsável por garantir que o software funcione corretamente.

Ele atua:

- criando cenários de teste
- executando testes
- identificando falhas

Tipos de teste incluem:

- unitário
- integração
- sistema

- regressão

Product Owner

Responsável por definir o que será desenvolvido.

Ele:

- prioriza tarefas
- representa o cliente
- define requisitos

Scrum Master

Responsável por garantir que o processo ágil funcione corretamente.

Ele:

- organiza reuniões
- remove impedimentos
- melhora o fluxo de trabalho

UX/UI Designer

Responsável pela experiência do usuário.

UX:

- usabilidade
- fluxo

UI:

- aparência
- design visual

3. LINGUAGENS DE PROGRAMAÇÃO

Uma linguagem de programação é um sistema formal que permite a comunicação entre o ser humano e o computador por meio de instruções que podem ser executadas. Essas instruções são escritas seguindo regras específicas chamadas de sintaxe, e cada instrução possui um significado que define seu comportamento, chamado de semântica.

As linguagens de programação existem porque o computador não entende diretamente linguagem humana. Ele executa operações em nível muito baixo, próximas de sinais elétricos e instruções de máquina. As linguagens servem como uma camada intermediária que torna possível expressar soluções de forma compreensível para humanos e executável para máquinas.

Cada linguagem foi criada com objetivos diferentes. Algumas priorizam desempenho, outras facilidade de uso, outras segurança, outras produtividade. Por isso, não existe uma linguagem universalmente melhor. A escolha depende do problema que se deseja resolver.

Por exemplo, linguagens como C são usadas em sistemas que precisam de controle direto de memória e alto desempenho. Linguagens como Python são usadas quando se deseja rapidez no desenvolvimento e facilidade de leitura. Linguagens como Java e C# são amplamente utilizadas em sistemas corporativos por oferecerem estrutura e robustez.

4. NÍVEL DE ABSTRAÇÃO DAS LINGUAGENS

O nível de abstração de uma linguagem indica o quanto ela se distancia dos detalhes internos do hardware. Quanto mais próxima do hardware, menor o nível de abstração. Quanto mais distante, maior o nível de abstração.

Linguagens de baixo nível são aquelas que possuem forte relação com o funcionamento interno do computador. Elas permitem controle direto sobre memória, registradores e instruções do processador. Esse tipo de linguagem exige conhecimento detalhado da arquitetura da máquina e costuma ser mais difícil de escrever e manter.

Assembly é o exemplo mais clássico desse tipo de linguagem. Nela, cada instrução corresponde diretamente a uma operação do processador. Isso

oferece controle total, mas torna o desenvolvimento mais lento e propenso a erros.

Linguagens de nível intermediário oferecem um equilíbrio entre controle e abstração. A linguagem C é um exemplo importante. Ela permite manipulação direta de memória, mas também oferece estruturas como funções, condicionais e loops, facilitando o desenvolvimento em comparação ao Assembly.

Linguagens de alto nível são aquelas que escondem a maior parte dos detalhes do hardware. Elas permitem que o programador se concentre mais na lógica do problema do que na forma como o computador executa cada instrução.

Python, Java, JavaScript e C# são exemplos de linguagens de alto nível. Nessas linguagens, tarefas complexas podem ser realizadas com poucas linhas de código, porque muitos detalhes já são gerenciados automaticamente.

O uso de linguagens de alto nível aumenta a produtividade e a legibilidade do código, mas reduz o controle direto sobre o hardware. Já linguagens de baixo nível oferecem controle máximo, mas com custo de complexidade.

Na prática, sistemas modernos combinam diferentes níveis. Partes críticas podem ser escritas em linguagens mais próximas do hardware, enquanto a maior parte do sistema é desenvolvida em linguagens de alto nível.

5. TIPAGEM EM LINGUAGENS DE PROGRAMAÇÃO

Tipagem é o conjunto de regras que define como os dados são classificados, armazenados e manipulados dentro de uma linguagem de programação. Um tipo representa uma categoria de valor, como números, textos, valores lógicos ou estruturas mais complexas.

Quando um valor é utilizado em um programa, ele possui um tipo. Esse tipo determina quais operações são válidas para aquele valor. Por exemplo, é possível somar dois números, mas não faz sentido somar diretamente um número com um texto sem alguma forma de conversão.

A tipagem influencia diretamente a segurança, a legibilidade e a manutenção do código. Linguagens com regras de tipagem mais rigorosas

tendem a evitar erros antes da execução. Linguagens mais flexíveis permitem maior liberdade, mas exigem mais cuidado do programador.

Existem duas formas principais de verificação de tipos.

Na tipagem estática, os tipos são verificados antes da execução do programa. Isso geralmente acontece durante a compilação. Se houver incompatibilidade entre tipos, o erro é detectado antecipadamente. Isso aumenta a segurança e facilita a manutenção em sistemas grandes.

Na tipagem dinâmica, os tipos são verificados durante a execução. Isso significa que um programa pode ser executado mesmo com possíveis inconsistências, e os erros só aparecem quando aquela parte do código é realmente executada. Esse modelo oferece mais flexibilidade, mas pode esconder erros até o momento da execução.

Outro aspecto importante é o grau de rigidez na manipulação dos tipos.

Em linguagens com regras mais rigorosas, operações entre tipos diferentes não são permitidas sem conversão explícita. Isso reduz ambiguidades e comportamentos inesperados.

Em linguagens mais flexíveis, a conversão entre tipos pode acontecer automaticamente. Isso pode facilitar o desenvolvimento em alguns casos, mas também pode gerar resultados inesperados se o programador não compreender bem as regras da linguagem.

A tipagem é importante porque afeta diretamente a forma como o programador pensa e escreve o código. Ela define o nível de controle, segurança e previsibilidade do sistema.

6. PARADIGMAS DE PROGRAMAÇÃO

Paradigmas de programação são formas de organizar e estruturar soluções computacionais. Eles representam diferentes maneiras de pensar sobre como um problema deve ser resolvido por meio de código.

Cada paradigma define uma abordagem específica para lidar com dados, lógica e fluxo de execução. Compreender paradigmas é fundamental porque permite escolher a melhor forma de resolver um problema, em vez de depender apenas de sintaxe de linguagem.

O paradigma imperativo é baseado na ideia de que um programa é uma sequência de instruções que alteram o estado do sistema. O programador define passo a passo o que deve ser feito. Esse modelo é muito próximo da forma como o computador executa operações.

O paradigma estruturado é uma evolução do modelo imperativo. Ele organiza o código em blocos bem definidos, utilizando sequência, decisões e repetições. Esse modelo melhora a legibilidade e evita estruturas desorganizadas.

O paradigma orientado a objetos organiza o software em torno de entidades chamadas objetos. Cada objeto possui dados e comportamentos. Esse modelo permite representar sistemas de forma mais próxima do mundo real, facilitando a organização de sistemas complexos.

Esse paradigma introduz conceitos como encapsulamento, que protege os dados internos de um objeto; abstração, que permite focar apenas nos aspectos essenciais; herança, que possibilita reutilização de estrutura; e polimorfismo, que permite tratar diferentes objetos de forma uniforme.

O paradigma funcional trata a computação como avaliação de funções. Ele evita a alteração de estado e utiliza funções que sempre produzem o mesmo resultado para as mesmas entradas. Isso torna o código mais previsível e facilita testes e execução paralela.

O paradigma declarativo se concentra em descrever o resultado desejado, sem especificar detalhadamente como alcançá-lo. O sistema é responsável por decidir como executar as instruções. Isso aumenta o nível de abstração e simplifica a escrita de código em determinados contextos.

O paradigma lógico é baseado em regras e relações. Em vez de escrever instruções passo a passo, o programador define fatos e regras, e o sistema realiza inferências para chegar a resultados.

Na prática, muitas linguagens modernas suportam múltiplos paradigmas. Isso permite que o programador combine diferentes abordagens dentro do mesmo sistema.

7. ARQUITETURA DE SOFTWARE

A arquitetura de software define a estrutura geral de um sistema. Ela descreve como os componentes são organizados, como se comunicam e como o sistema atende aos requisitos funcionais e não funcionais.

A arquitetura é importante porque decisões estruturais têm impacto direto na escalabilidade, manutenção, desempenho e segurança do sistema.

Um sistema monolítico é aquele em que todas as funcionalidades estão integradas em uma única aplicação. Essa abordagem é simples e fácil de iniciar, mas pode se tornar difícil de manter conforme o sistema cresce.

A arquitetura em camadas organiza o sistema em níveis de responsabilidade. Cada camada possui uma função específica, como interface, lógica de negócio e acesso a dados. Isso melhora a organização e facilita manutenção.

A arquitetura baseada em microserviços divide o sistema em serviços menores e independentes. Cada serviço possui responsabilidade específica e pode ser desenvolvido e implantado separadamente. Isso permite escalabilidade e flexibilidade, mas aumenta a complexidade.

A escolha da arquitetura depende do tamanho do sistema, da equipe, dos requisitos e da necessidade de evolução.

8. APIs

Uma API é uma interface que permite a comunicação entre sistemas. Ela define como um sistema pode solicitar serviços ou dados de outro sistema.

APIs são fundamentais porque permitem integração entre diferentes aplicações. Por exemplo, um sistema pode utilizar uma API para processar pagamentos, enviar mensagens ou acessar dados de outro serviço.

Na web, APIs geralmente utilizam o protocolo HTTP. Métodos como GET, POST, PUT e DELETE definem o tipo de operação realizada.

APIs bem projetadas são claras, consistentes e seguras. Elas permitem que diferentes sistemas trabalhem juntos de forma organizada.

9. BANCO DE DADOS

Bancos de dados são sistemas responsáveis por armazenar, organizar e recuperar informações.

Bancos relacionais organizam dados em tabelas. Cada tabela possui colunas e linhas, e os dados podem ser relacionados entre si. Esse modelo é baseado em regras rígidas e é muito utilizado em sistemas que exigem consistência.

Bancos não relacionais utilizam estruturas mais flexíveis, como documentos ou pares chave-valor. Eles são usados em cenários que exigem escalabilidade e flexibilidade.

A escolha do banco depende do tipo de aplicação, volume de dados e forma de acesso.

10. VERSIONAMENTO DE CÓDIGO

Versionamento é o controle das alterações feitas em um sistema ao longo do tempo.

Ferramentas como Git permitem registrar cada mudança, possibilitando:

- histórico completo
- reversão de alterações
- trabalho em equipe

O versionamento é essencial em projetos reais, pois evita perda de código e facilita colaboração entre desenvolvedores.

11. STACK DE TECNOLOGIA

Stack de tecnologia é o conjunto de ferramentas e tecnologias utilizadas para construir um sistema.

Uma stack normalmente inclui:

- linguagem de programação

- framework
- banco de dados
- servidor
- infraestrutura

Um exemplo clássico é a stack LAMP, composta por Linux, Apache, MySQL e PHP.

Outro exemplo é a stack MERN, composta por MongoDB, Express, React e Node.js.

A stack representa todas as camadas de uma aplicação, desde a interface até o armazenamento de dados.

12. FUNDAMENTOS COMPUTACIONAIS

Os fundamentos computacionais são a base da programação.

Algoritmos são sequências de passos que resolvem problemas. Eles definem a lógica do sistema.

Estruturas de dados organizam informações de forma eficiente. Exemplos incluem listas, filas e árvores.

A complexidade mede o desempenho de algoritmos. A notação Big O descreve como o tempo de execução cresce com o tamanho da entrada.

Esses conceitos são essenciais para desenvolver soluções eficientes.

13. CONCLUSÃO

A Engenharia de Software envolve uma combinação de conhecimentos técnicos, práticas organizacionais e pensamento estruturado.

Compreender linguagens, paradigmas, arquitetura, dados e processos permite que o profissional construa sistemas de forma eficiente e sustentável.

Esse conjunto de conhecimentos forma a base necessária para evoluir na área de tecnologia.



14. PRINCIPAIS SOFTWARES, FERRAMENTAS E TECNOLOGIAS DA ENGENHARIA DE SOFTWARE

O desenvolvimento de software moderno depende de um ecossistema amplo de ferramentas que atuam em diferentes etapas do ciclo de vida de um sistema. Essas ferramentas não são utilizadas de forma isolada, mas sim combinadas em conjuntos que permitem desenvolver, executar, testar, implantar e manter aplicações de forma eficiente.

A compreensão dessas tecnologias é essencial para entender como sistemas reais são construídos na prática.

14.1 AMBIENTES DE DESENVOLVIMENTO

Ambientes de desenvolvimento são ferramentas utilizadas para escrever, organizar, executar e depurar código. Eles aumentam significativamente a produtividade do desenvolvedor ao oferecer recursos como autocompletar, análise de erros e integração com outras ferramentas.

Visual Studio Code

Empresa: Microsoft

Ano: 2015

Link: <https://code.visualstudio.com>

O Visual Studio Code é um editor de código altamente extensível e leve, amplamente adotado na indústria. Seu diferencial está na capacidade de se adaptar a diferentes linguagens e contextos por meio de extensões.

Ele permite ao desenvolvedor trabalhar com diversas tecnologias em um único ambiente, oferecendo suporte a depuração, integração com Git e execução de código diretamente no editor.

É utilizado principalmente em:

- desenvolvimento web
- APIs
- scripts e automação
- projetos multi-linguagem

Empresas que utilizam essa tecnologia:

- Microsoft, em desenvolvimento interno
 - Google, em projetos diversos
 - startups e empresas modernas em geral
-

IntelliJ IDEA

Empresa: JetBrains

Ano: 2001

Link: <https://www.jetbrains.com/idea>

IntelliJ IDEA é uma IDE completa voltada para desenvolvimento Java e ecossistema JVM. Ela é conhecida por sua capacidade de análise profunda de código e ferramentas avançadas de refatoração.

Esse ambiente auxilia o desenvolvedor a escrever código mais seguro e organizado, reduzindo erros e melhorando a manutenção de sistemas complexos.

É utilizada principalmente em:

- sistemas corporativos
- aplicações financeiras
- projetos de grande escala

Empresas que utilizam essa tecnologia:

- bancos como Itaú e Bradesco
 - empresas de sistemas corporativos
-

PyCharm

Empresa: JetBrains

Ano: 2010

Link: <https://www.jetbrains.com/pycharm>

PyCharm é uma IDE especializada em Python, oferecendo suporte avançado para desenvolvimento web, ciência de dados e automação.

Ela inclui ferramentas específicas para análise de código Python, integração com frameworks e suporte a ambientes virtuais.

É utilizada principalmente em:

- projetos de ciência de dados
- inteligência artificial
- back-end em Python

Empresas que utilizam essa tecnologia:

- empresas de tecnologia e dados
 - equipes que trabalham com machine learning
-

Eclipse

Empresa: Eclipse Foundation

Ano: 2001

Link: <https://www.eclipse.org>

Eclipse é uma IDE tradicional e open-source, amplamente utilizada no desenvolvimento Java. Possui arquitetura modular baseada em plugins.

É utilizada principalmente em:

- sistemas legados
- ambientes corporativos tradicionais

Empresas que utilizam essa tecnologia:

- instituições governamentais

- empresas com sistemas antigos
-

Visual Studio

Empresa: Microsoft

Ano: 1997

Link: <https://visualstudio.microsoft.com>

Visual Studio é uma IDE robusta voltada para o desenvolvimento com tecnologias Microsoft, especialmente C# e .NET.

Oferece ferramentas completas para criação de aplicações desktop, web e serviços backend.

É utilizada principalmente em:

- aplicações corporativas
- sistemas Windows
- soluções empresariais

Empresas que utilizam essa tecnologia:

- empresas do setor corporativo
 - organizações que utilizam stack Microsoft
-

14.2 CONTROLE DE VERSÃO E REPOSITÓRIOS

Ferramentas de controle de versão permitem rastrear alterações no código e possibilitam o trabalho colaborativo entre desenvolvedores.

Git

Criado por Linus Torvalds

Ano: 2005

Link: <https://git-scm.com>

Git é um sistema de controle de versão distribuído que permite registrar o histórico completo de alterações de um projeto.

Ele possibilita a criação de ramificações independentes, permitindo que múltiplos desenvolvedores trabalhem simultaneamente sem conflitos diretos.

É utilizado em praticamente todos os projetos de software modernos.

Empresas que utilizam essa tecnologia:

- todas as empresas de tecnologia, sem exceção significativa
-

GitHub

Empresa: Microsoft

Ano: 2008

Link: <https://github.com>

GitHub é uma plataforma baseada em Git que permite hospedar repositórios e colaborar em projetos.

Ele facilita revisão de código, automação e integração com diversas ferramentas.

Empresas que utilizam essa tecnologia:

- Microsoft
 - Meta
 - Netflix
-

GitLab

Empresa: GitLab Inc.

Ano: 2011

Link: <https://gitlab.com>

GitLab é uma plataforma completa que integra repositórios, pipelines de automação e ferramentas de deploy.

É utilizada principalmente quando se deseja centralizar todo o ciclo de desenvolvimento em um único ambiente.

Empresas que utilizam essa tecnologia:

- IBM
 - empresas com infraestrutura própria
-

Bitbucket

Empresa: Atlassian

Ano: 2008

Link: <https://bitbucket.org>

Bitbucket é uma plataforma de versionamento integrada ao ecossistema Atlassian.

É utilizada principalmente por equipes que utilizam ferramentas como Jira.

Empresas que utilizam essa tecnologia:

- empresas que utilizam Jira e ferramentas Atlassian
-

14.3 GERENCIADORES DE PACOTES

Gerenciadores de pacotes são ferramentas responsáveis por instalar, atualizar e gerenciar dependências de um projeto.

npm

Linguagem: JavaScript

Empresa: npm Inc. / Microsoft

Ano: 2010

Link: <https://www.npmjs.com>

npm é o principal gerenciador de pacotes do ecossistema JavaScript. Ele permite instalar bibliotecas e automatizar tarefas de desenvolvimento.

Empresas que utilizam essa tecnologia:

- Netflix
 - Uber
 - PayPal
-

yarn

Linguagem: JavaScript

Empresa: Meta

Ano: 2016

Link: <https://yarnpkg.com>

Yarn é uma alternativa ao npm com foco em desempenho e consistência na instalação de dependências.

Empresas que utilizam essa tecnologia:

- Meta
 - empresas que trabalham com aplicações de grande escala em JavaScript
-

pip

Linguagem: Python

Ano: 2008

Link: <https://pypi.org>

pip é o gerenciador de pacotes padrão do Python.

Empresas que utilizam essa tecnologia:

- Instagram
 - Spotify
-

apt

Sistema: Linux

Ano: 1998

Link: <https://wiki.debian.org/Apt>

apt é um gerenciador de pacotes de sistemas Linux baseados em Debian.

Ele é utilizado para instalar softwares no sistema operacional.

Empresas que utilizam essa tecnologia:

- empresas que utilizam servidores Linux

Homebrew

Sistema: macOS

Ano: 2009

Link: <https://brew.sh>

Homebrew é um gerenciador de pacotes para macOS, utilizado para instalar ferramentas de desenvolvimento.

Empresas que utilizam essa tecnologia:

- desenvolvedores que utilizam macOS

Maven

Linguagem: Java

Empresa: Apache

Ano: 2004

Link: <https://maven.apache.org>

Maven é uma ferramenta de automação de build e gerenciamento de dependências.

Empresas que utilizam essa tecnologia:

- sistemas corporativos Java

Gradle

Linguagem: Java/Kotlin

Ano: 2007

Link: <https://gradle.org>

Gradle é uma ferramenta moderna de build que oferece maior flexibilidade que o Maven.

Empresas que utilizam essa tecnologia:

- empresas que utilizam Android e Java moderno
-

14.4 CONTAINERS E INFRAESTRUTURA

Docker

Empresa: Docker Inc.

Ano: 2013

Link: <https://www.docker.com>

Docker permite empacotar aplicações com todas as suas dependências em containers isolados.

Isso garante que a aplicação funcione da mesma forma em qualquer ambiente.

Empresas que utilizam essa tecnologia:

- Spotify
- Amazon
- Airbnb

Kubernetes

Empresa: Google

Ano: 2014

Link: <https://kubernetes.io>

Kubernetes é uma plataforma de orquestração de containers que permite gerenciar aplicações distribuídas.

Ele automatiza tarefas como escalabilidade, recuperação e balanceamento.

Empresas que utilizam essa tecnologia:

- Google
 - Netflix
 - Amazon
-

14.5 SERVIDORES E EXECUÇÃO

Node.js

Empresa: OpenJS Foundation

Ano: 2009

Link: <https://nodejs.org>

Node.js permite executar JavaScript no servidor, utilizando um modelo assíncrono eficiente para lidar com múltiplas conexões.

É amplamente utilizado na construção de APIs e aplicações em tempo real.

Empresas que utilizam essa tecnologia:

- Netflix
 - LinkedIn
 - PayPal
-

Nginx

Empresa: Nginx Inc.

Ano: 2004

Link: <https://nginx.org>

Nginx é um servidor web de alta performance utilizado também como proxy reverso.

Ele é capaz de lidar com milhares de conexões simultâneas com baixo consumo de recursos.

Empresas que utilizam essa tecnologia:

- Netflix
 - Dropbox
 - WordPress
-

Apache HTTP Server

Empresa: Apache Software Foundation

Ano: 1995

Link: <https://httpd.apache.org>

Apache é um servidor web tradicional amplamente utilizado em sistemas antigos e aplicações clássicas.

Empresas que utilizam essa tecnologia:

- organizações com sistemas legados
 - instituições governamentais
-

14.6 BANCOS DE DADOS

MySQL

Empresa: Oracle

Ano: 1995

Link: <https://www.mysql.com>

MySQL é um banco de dados relacional amplamente utilizado em aplicações web.

Empresas que utilizam essa tecnologia:

- Wikipedia
 - aplicações web tradicionais
-

PostgreSQL

Ano: 1996

Link: <https://www.postgresql.org>

PostgreSQL é um banco relacional avançado, conhecido por sua confiabilidade.

Empresas que utilizam essa tecnologia:

- Apple
- empresas financeiras

MongoDB

Empresa: MongoDB Inc.

Ano: 2009

Link: <https://www.mongodb.com>

MongoDB é um banco NoSQL que armazena dados em formato flexível.

Empresas que utilizam essa tecnologia:

- Uber
- eBay

Redis

Empresa: Redis Ltd.

Ano: 2009

Link: <https://redis.io>

Redis é um banco de dados em memória usado para cache e alta performance.

Empresas que utilizam essa tecnologia:

- Twitter
- GitHub

14.7 FRAMEWORKS

React

Empresa: Meta

Ano: 2013

Link: <https://react.dev>

React é uma biblioteca para construção de interfaces baseadas em componentes reutilizáveis.

Empresas que utilizam essa tecnologia:

- Facebook

- Instagram
 - Airbnb
-

Angular

Empresa: Google

Ano: 2016

Link: <https://angular.io>

Angular é um framework completo para desenvolvimento front-end.

Empresas que utilizam essa tecnologia:

- Google
 - empresas corporativas
-

Django

Empresa: Django Software Foundation

Ano: 2005

Link: <https://www.djangoproject.com>

Django é um framework web completo para Python, focado em produtividade e segurança.

Empresas que utilizam essa tecnologia:

- Instagram
 - Pinterest
 - Mozilla
-

Spring Boot

Empresa: VMware

Ano: 2014

Link: <https://spring.io>

Spring Boot é um framework Java voltado para criação de aplicações backend robustas.

Empresas que utilizam essa tecnologia:

- bancos
- empresas corporativas globais

CONCLUSÃO

As ferramentas apresentadas formam a base do ecossistema moderno de desenvolvimento de software. Cada uma desempenha um papel específico dentro do ciclo de vida das aplicações, desde a escrita do código até sua execução em produção.

Compreender essas tecnologias permite ao estudante visualizar como sistemas reais são construídos e mantidos, conectando teoria e prática dentro da Engenharia de Software.